

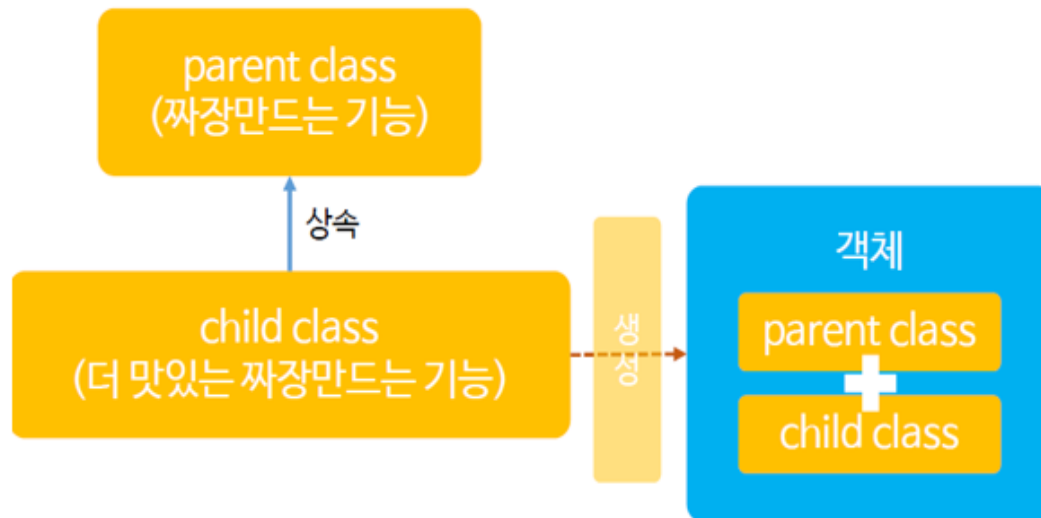
17강

메서드 오버라이딩 및 다형성





- 부모 클래스의 기능을 자식 클래스에서 재정의 해서 사용한다.



```
package lec19Pjt001;

public class ParentClass {

    public void makeJJajang() {
        System.out.println(" -- makeJJajang() START --");
    }
}
```

```
package lec19Pjt001;

public class ChildClass extends ParentClass {

    @Override
    public void makeJJajang() {
        System.out.println(" -- more delicious makeJJajang() START --");
    }
}
```

```
ChildClass child = new ChildClass();
child.makeJJajang();
```

```
<terminated> MainClass (16) [Java Application] C:\Program Files\Java
ParentClass constructor
ChildClass constructor
-- more delicious makeJJajang() START --
```



오버라이딩은 부모클래스에 있는 메소드를 자식클래스에서 재정의하여 사용하는 것을 말합니다.

따라서 오버라이딩은 메소드의 이름 및 매개변수의 타입과 개수 등은 모두 동일하고 내용만 변경하여 사용됩니다.

(단, 접근지정자의 경우, 자식클래스의 메소드 범위가 부모클래스의 메소드 범위보다 넓거나 같아야합니다.)

```
public class Book {
```

```
    int price = 1000;
```

```
    public void showPrice() {  
        System.out.println("가격:" + price);  
    }
```

```
}
```

부모 클래스

재정의

```
public class EnglishBook extends Book {
```

```
    public void showPrice() {  
        System.out.println("영어책 가격:" + price);  
    }
```

```
}
```

자식 클래스

오버라이딩

<https://kadosholy.tistory.com>



오버로딩은 같은 이름의 메소드를 매개변수의 타입이나 개수에 따라서 중복 정의함으로써, 매개변수에 따라 특정 메소드가 호출되도록 하는 것을 말합니다.

```
public class EnglishBook extends Book {
```

```
    public void showPrice() {  
        System.out.println("영어책 가격:" + price);  
    }
```

매개변수가 다른
동일 이름의 메소드

```
    public void showPrice(int count) {  
        System.out.println("영어책 " + count + "권 가격:" + price * count);  
    }
```

오버로딩

```
}
```

<https://kadosholy.tistory.com>

오버로딩 (Overloading)



➤ 기본 자료형처럼 클래스도 자료형이다.

```
package lec19Pjt001;

public class ParentClass {
    public void makeJJajang() {
        System.out.println(" -- makeJJajang() START --");
    }
}
```

```
package lec19Pjt001;

public class FirstChildClass extends ParentClass {

    @Override
    public void makeJJajang() {
        System.out.println(" -- FirstChildClass's makeJJajang() START --");
    }
}
```

```
package lec19Pjt001;

public class SecondChildClass extends ParentClass {

    @Override
    public void makeJJajang() {
        System.out.println(" -- SecondChildClass's makeJJajang() START --");
    }
}
```

```
ParentClass childs[] = new ParentClass[2];
childs[0] = new FirstChildClass();
childs[1] = new SecondChildClass();

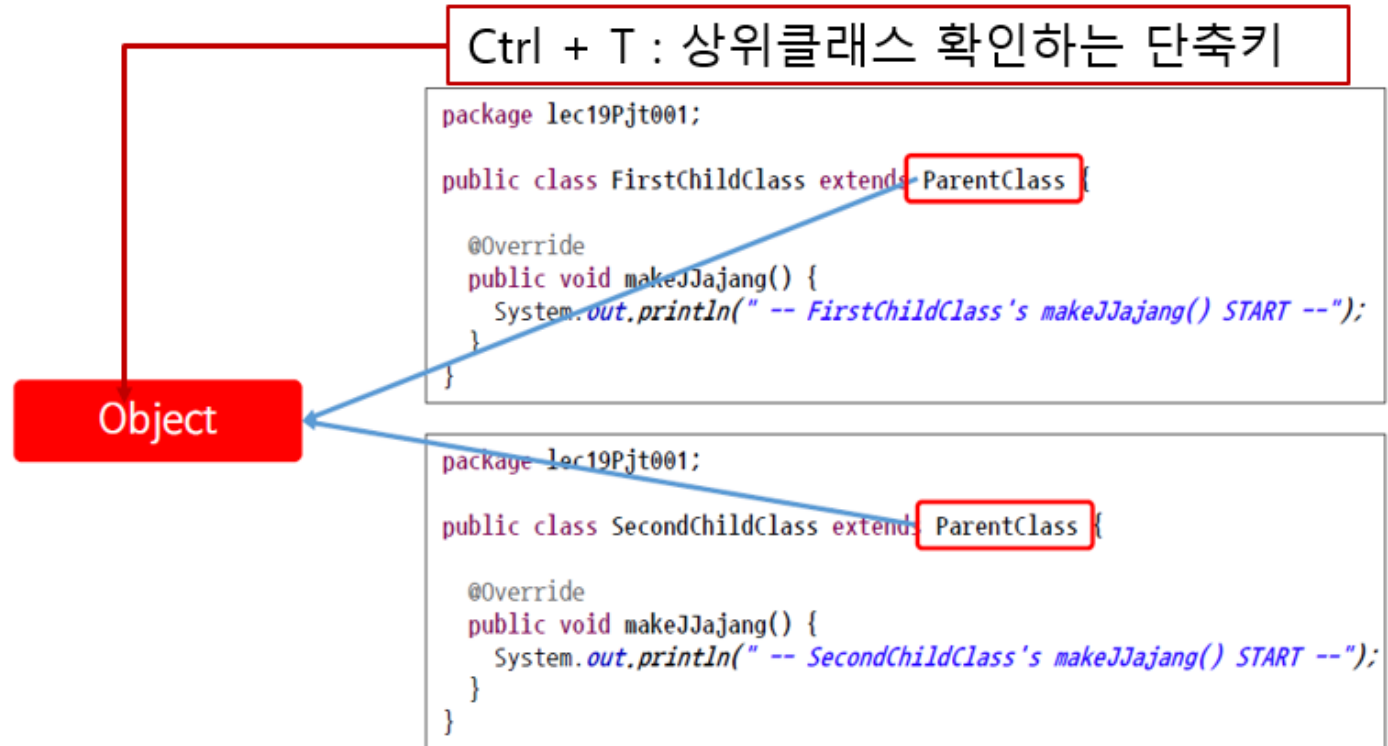
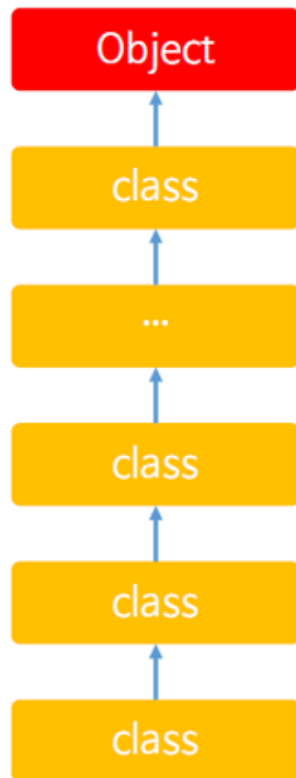
for (int i = 0; i < childs.length; i++) {
    childs[i].makeJJajang();
}
```



```
ParentClass constructor
FirstChildClass constructor
ParentClass constructor
SecondChildClass constructor
-- FirstChildClass's makeJJajang() START --
-- SecondChildClass's makeJJajang() START --
```



- 모든 클래스의 최상위 클래스는 Object 클래스이다.



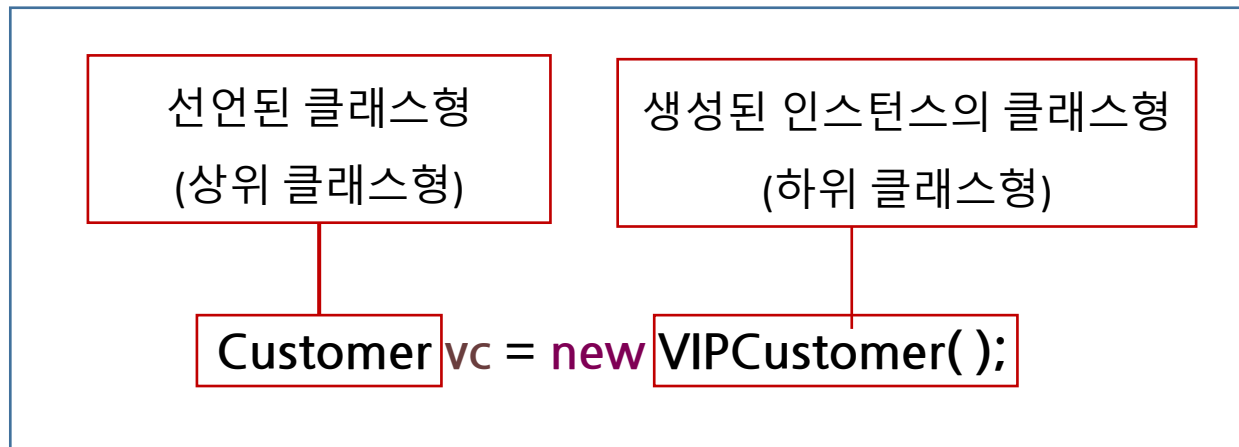


➤ 캐스팅(Casting)이란

캐스팅이란 **타입을 변환하는 것**을 말하며 **형 변환**이라고도 한다. 자바의 상속 관계에 있는 부모와 자식 클래스 간에는 서로 간의 형 변환이 가능하다.

➤ 상위 클래스로의 묵시적 형 변환(업캐스팅)이란

- 상위 클래스 형으로 변수를 선언하고 하위 클래스 인스턴스를 생성 할 수 있음
- 하위 클래스는 상위 클래스의 타입을 내포하고 있으므로 상위 클래스로 묵시적 형 변환이 가능 함





➤ 묵시적 형 변환과 재정의된 메서드 호출

```
Customer vp = new VIPCustomer();
vp.calcprice(10000);
```

위 코드에서 calcprice() 메서드는 어느 메서드가 호출 될 것인가 ?

```
1 package inheritance4;
2 import java.text.*;
3 public class CustomerMain {
4
5     public static void main(String[] args) {
6
7         DecimalFormat de = new DecimalFormat("#,###원");
8
9         Customer vc = new VIPCustomer(0200, "김길동", 1010);
10        int price = vc.calcprice(10000);
11        System.out.println("지불 금액은 " + de.format(price) + "이고, " + vc.ShowCustomerInfo());
12    }
```

Console ×

<terminated> CustomerMain (1) [Java Application] C:\Users\Admin\p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64

지불 금액은 9,000원이고,김길동님의 등급은 VIP이며, 보너스 포인트는 500입니다.



➤ 가상 메서드(virtual method)

프로그램에서 어떤 객체의 변수나 메서드의 참조는 그 타입에 따라 이루어짐, 가상 메서드의 경우는 타입과 상관없이 실제 생성된 인스턴스의 메서드가 호출 되는 원리
자바는 모든 메서드가 가상 메서드이다.

가상 메서드에의해서 다형성이 구현된다

```
Customer vc = new VIPCustomer();
```

```
vc.calcprice(10000);
```

vc의 타입은 Customer 지만, 실제 생성된 인스턴스인 VIPCustomer 클래스의 calcprice()메서드가 호출됨.

=> 이렇게 호출되는 것을 메서드 바인딩이라 부른다.



➤ 가상 메서드(virtual method)

재정의의 → 가상메서드로 감

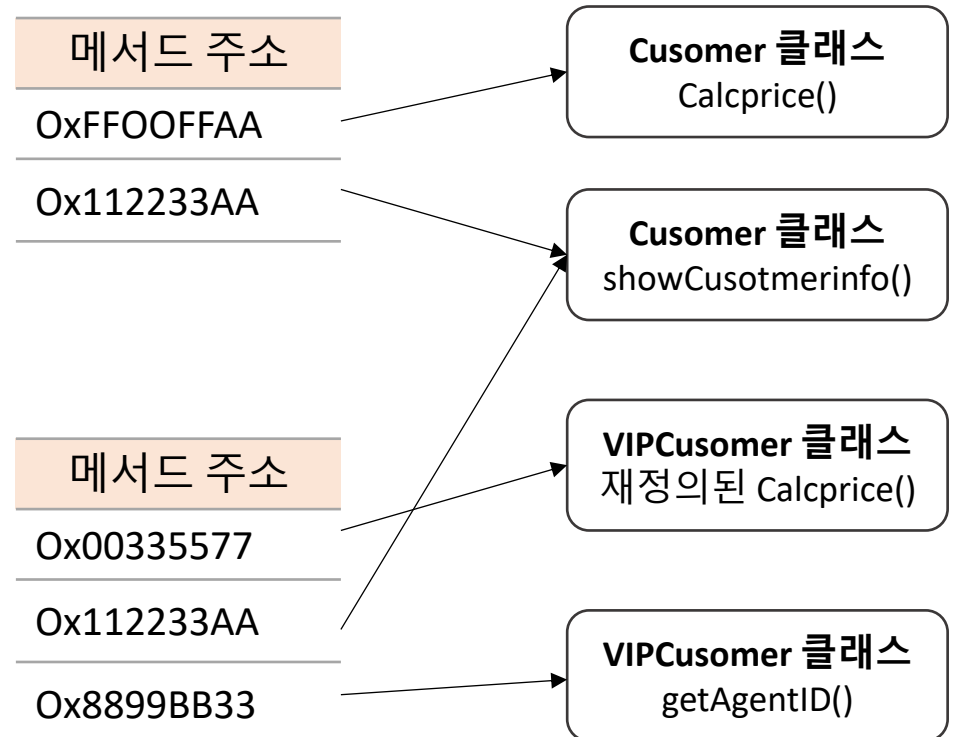
Customer 클래스의 가상 메서드 테이블

메서드
Calcprice(재정의됨)
showCustomerinfo(재정의되지 않음)

VIPCustomer 클래스의 가상 메서드 테이블

메서드
Calcprice(재정의됨)
showCustomerinfo(재정의되지 않음)
getAgentID(하위클래스에서 추가된 메서드)

메서드 영역





➤ 다형성(polymorphism)

하나의 코드가 여러가지 자료형으로 구현되어 실행되는 것

정보은닉, 상속 과 더불어 객체지향 프로그래밍의 가장 큰 특징 중 하나

객체지향 프로그래밍의 유연성, 재활용성, 유지보수성에 기본이 되는 특징임.

다형성 성립 조건

1. 상속 관계

2. 메서드 오버라이딩

3. 부모 타입으로 자식 객체 참고



```
package polymorphism;
```

```
class Animal {  
    public void move() {  
        System.out.println("동물이 움직입니다.");  
    }  
}  
  
class Hauman extends Animal {  
    public void move() {  
        System.out.println("사람은 두발로 걷습니다.");  
    }  
}  
  
class Tiger extends Animal {  
    public void move() {  
        System.out.println("호랑이가 네발로 뜀니다.");  
    }  
}  
  
class Eagle extends Animal {  
    public void move() {  
        System.out.println("독수리가 하늘을 날읍니다.");  
    }  
}
```

```
public class AnimalTest {
```

```
    public static void main(String[] args) {  
        AnimalTest atest = new AnimalTest();  
        atest.moveAnimal(new Hauman());  
        atest.moveAnimal(new Tiger());  
        atest.moveAnimal(new Eagle());  
    }  
  
    public void moveAnimal(Animal animal) {  
        animal.move();  
    }  
}
```

Console X
<terminated> AnimalTest [Java Application] C:\WUs
사람은 두발로 걷습니다.
호랑이가 네발로 뜀니다.
독수리가 하늘을 날읍니다.

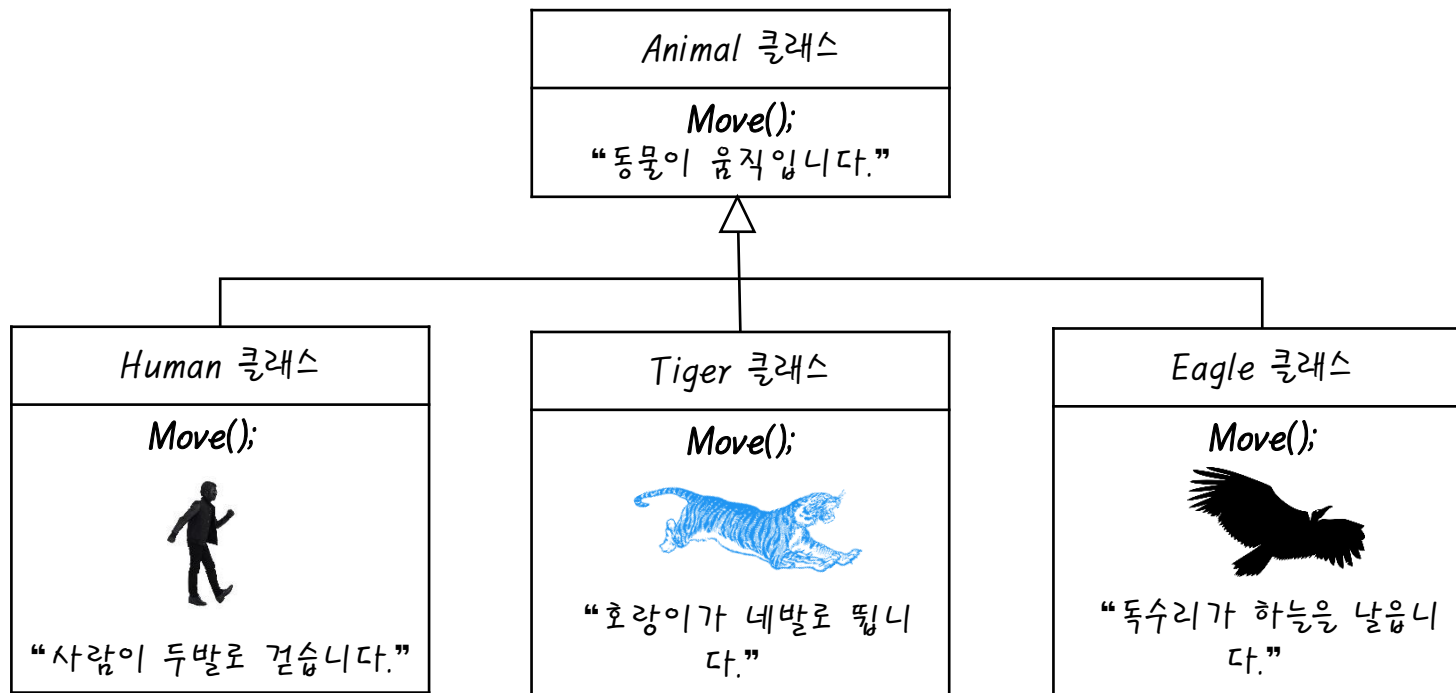


➤ 다형성 구현 하기

하나의 클래스를 상속 받은 여러 클래스가 있는 경우

각 클래스마다 같은 이름의 서로 다른 메서드를 재정의 함

상위 클래스 타입으로 선언된 하나의 변수가 여러 인스턴스에 대입되어 다양한 구현이 실행 될 수 있음





➤ 다형성 구현 하기

```
public class AnimalTest {  
  
    public static void main(String[] args) {  
        AnimalTest atest = new AnimalTest();  
        atest.moveAnimal(new Hauman());  
        atest.moveAnimal(new Tiger());  
        atest.moveAnimal(new Eagle());  
    }  
  
    public void moveAnimal(Animal animal) {  
        animal.move();  
    }  
}
```

동일한 animal.move()
코드에 대해 각기 다른
구현이 실행 됨



Console X
<terminated> AnimalTest [Java Application] C:\WUs
사람은 두발로 걷습니다.
호랑이가 네발로 뜀니다.
독수리가 하늘을 날읍니다.

매개변수의 자료형이 상위 클래스

재정의된 메서드가 호출됨